

设计文档：ClosedLoop 执行型本地生活 Agent

TL;DR

ClosedLoop 是一个执行型本地生活 Agent：用户一句话 → 生成 4-6 小时行程（玩→吃→后续）→ 用户确认 → 生成待支付执行命令前先做一致性校验与必要补齐（fixup）→ 校验通过后生成待支付执行命令 → 输入 Mock 支付密码 111111 → 付款后 Mock commit。
设计目标不是“最炫”，而是**响应快、稳定可复现、能落地**（单工具 ≤ 3s，规划 ≤ 30s，端到端 ≤ 2min；失败可恢复且状态不污染）。

背景（S）

本赛题要求在周末 4-6 小时窗口内完成“规划 → 确认 → Mock 执行”的闭环，并满足严格时延预算与稳定验收（可复现、失败可恢复）。

难点（C）

- 规划侧：端到端 LLM 难稳定满足强时空/预算约束，且多轮工具补算易超时，输出不确定。
- 执行侧：确认执行后不会立刻让用户付款，而是先做执行前一致性校验与必要补齐（fixup）；只有在库存、座位与备选问题处理完成后，才生成待支付命令。这样可以避免用户先付款、后补齐，进而减少退款或二次支付。
- 工程侧：在严格时延预算下，还要做到超时/降级/可恢复错误，且保证状态不污染、结果可复现。

关键问题（Q）

如何在严格时延预算下，把“多约束规划 + Mock 执行 + 失败自愈”做成稳定、可复现、可快速验收的闭环？

方案（A：Planning 策略）

- Planning 策略（确定性骨架）**：确定性模块负责候选召回/过滤/排序与时空预算校验，LLM 只做语义约束抽取与解释协商（细节与工程证据见 02_tools.md）。
- 单一入口 + 轻量补齐：首次规划/改约束统一走 plan_trip；执行失败优先静默等价替换，必须打扰时进入 fixup_agent 给 1-2 个选项继续推进。
- 可复现工程化：付款前只做可用性检查与命令生成；输入 Mock 支付密码 111111 后才提交 Mock commit，避免付款前副作用污染（工程证据详见 02_tools.md）。

适用范围与能力边界

当前 Demo 聚焦于本地生活聚会场景下的“意图理解 → 结构化约束 → 候选召回/过滤/重排 → 行程规划 → Mock 执行”闭环展示。系统当前更适合处理 family / friends 两类常见群体、4-6 小时窗口以及常见预算、距离、饮食、活动偏好组合下的问题。

需要明确的是，当前实现并不承诺所有输入都存在可行解，也不承诺所有忌口、偏好与极端限制能够同时满足。代码层面，plan_trip 接收的是结构化约束而非开放域自由规划；规划子图也采用确定性的召回、过滤、重排与组合剪枝流程，因此当候选不足、预算过紧、时长冲突或约束组合过强时，系统会明确返回无可行方案或可恢复错误，而不是伪造结果。

当前版本中，若从 plan_trip 的结构化输入契约来看，已经显式离散支持的关键约束包括：

- 群体类型：family、friends
- 距离偏好：<2km、2km-5km、>5km
- 出行偏好：auto、walking、taxi、driving
- 排队偏好：avoid_queues、accept_hot、neutral
- 礼品开关：include_gift = true / false

同时，当前版本也支持若干半结构化或有限开放集合的约束输入，例如：

- 饮食禁忌与偏好：优先归类到 辣 / 海鲜 / 生冷 / 甜 / 快餐 / 牛，不在集合内时可保留原词

- 活动偏好：通过 activity_preferences 传入，如 餐饮 / 电影 / 打卡点 / 安静
- 行程顺序偏好：通过 preferred_pattern_steps 指定步骤序列，系统会优先尝试匹配或构造对应 pattern

在当前版本中，部分推荐、规划与语义理解能力采用规则、契约归一化和 workflow 编排进行可复现近似。这一选择并不否认相关问题的复杂性，而是为了在比赛 Demo 的抽象层级下，把真实业务中已被广泛验证的搜索、推荐、语义提取与执行编排能力，以可解释、可验收、可复现的方式映射到闭环系统中。

工具调用链路（用户动作闭环）

- 规划：plan_trip → itinerary + candidates（候选池用于后续搜索/替换）。
- 修改/多看/替换：改约束仍走 plan_trip；想多看走 generate_alternative_plans；指定替换走 search_candidates → adjust_plan_item。
- 确认执行：transfer_to_execute → execute_itinerary；若执行前发现库存/座位/一致性问题，则先进入 fixup_agent，由用户选候选或搜索后走 adjust_and_execute_plan_item；只有补齐完成且一致性校验通过后，才会生成待支付执行命令并在输入 111111 后进入 mock_executor commit。

异常覆盖（3类）

1) 执行失败（无座/无票/库存不足）

- 失败分层与低打扰推进：若该 step 未被用户指定/锁定（user_touched=false，适用于活动/餐厅/礼物），且备选不会导致超预算/超时或明显偏好受损（requires_confirmation=false），则静默替换并继续执行；否则进入 confirmation.status=needs_fixup，由 fixup_agent 给 1-2 个选项并继续推进（本版本主要支持 equivalent_only；strict/completion_first 暂未在全链路启用）。
- 备选自动替换（等价优先）：在准备生成待支付执行单时，会先检查 combo/package 还能不能下单；如果发现不可用，就优先使用规划阶段为每个 step 预生成的 2 个 backup_candidates 做等价替换（默认策略 replacement_policy=equivalent_only）。
- 工程兜底：付款前不写运行时数据；Mock 支付密码校验通过后才进入 commit，若提交期一致性不满足则返回可重试错误码。

2) 用户修改导致冲突（替换后超窗/超预算）

- 多层降级与冲突修复：当用户替换条目导致时间或预算超标时，底层触发多层压缩（吃 Buffer -> 极限压缩体验时间）。
- 权衡报告 (Trade-off Report)：一般情况下，普通的等价替换不会超出用户预期；但当替换可能导致时间或预算超标时，系统会尽可能地通过多层降级进行权衡，进而不超出相关约束。当触发了这种降级权衡时，替换工具才会将计算出的明细通过 tradeoff_report 回传。此时 Agent 会在对话中向用户如实转达（例如：“为您替换了该活动，但为了不超出总时长，吃掉了 15 分钟缓冲时间并极限压缩了活动时长”），让调整后果完全透明，而不是静默强行写入或简单报错。

3) 无可行方案（0 候选 / 规划失败）

- 当硬约束过强导致无法产出方案，系统返回可恢复错误，并引导用户放宽最小必要约束（预算、距离、窗口、偏好等）。
- 当规划子服务不可用/超时（≤3s），主服务返回 TOOL_TIMEOUT，不中断对话，可重试或改用再规划。

取舍说明（补充）

- 相比纯 LLM 端到端：确定性骨架兜底可行性与稳定性，避免多轮补算导致等待变长与输出不稳。
- 相比过细微服务拆分：仅保留主链路必要的服务边界，减少接口契约与排障成本，优先保证可验收。
- 工程证据（失败码/验收动作/实现索引/付款门控一致性）统一收敛在 02_tools.md，design 只保留闭环叙事口径。